

CrediX: A Credit-Driven Distributed Buffer Management for Large-Scale Switching Chips

Xu Wang¹, Yang Zhang², Danfeng Shan³, Fengyuan Ren^{1,2*}

¹Lanzhou University, ²Tsinghua University, ³Xi'an Jiaotong University

Abstract

With the scaling of switching chips, modular architectures are becoming mainstream. As a result, buffers become physically distributed across tiles, leading to poor utilization under imbalanced traffic. To address this, we propose CrediX, a lightweight dynamic buffer management mechanism that aggregates distributed buffers into region-based shared pools and allocates usage on demand through a Regional Credit Allocator (RCA). The design leverages existing credit-based flow control and preserves packet ordering via a simple VC-to-path mapping. Evaluations show that CrediX reduces backpressure by 65.6% under synthetic Hotspot traffic and mitigates severe hotspot occupancy in a realistic workload.

Keywords

NoC, Switching Chip, Distributed Architecture, Buffer Management.

ACM Reference Format:

Xu Wang¹, Yang Zhang², Danfeng Shan³, Fengyuan Ren^{1,2}. 2026. CrediX: A Credit-Driven Distributed Buffer Management for Large-Scale Switching Chips. In *63rd ACM/IEEE Design Automation Conference (DAC '26)*, July 26–29, 2026, Long Beach, CA, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3770743.3804045>

1 Introduction

Large-scale AI models, such as Large Language Models (LLMs), rely on massive compute clusters for training and inference. As model scale and computational tasks increase, the demand for inter-node communication rises sharply, especially during parallel processing and synchronization, leading to frequent high-bandwidth data transfers[34]. Consequently, this expansion introduces significant communication bottlenecks[14], severely impacting the efficiency of distributed training and overall system performance. To mitigate this bottleneck, the switch chip, as the core switching component, must support a higher radix and massive bandwidth. This design enables flatter network topologies[15, 26, 28, 36], which reduce network diameter and communication latency.

To adapt to the growing demands for bandwidth and scalability, chip architectures are gradually evolving from monolithic to modular forms. The traditional monolithic switch architecture[13]

relies on a centralized shared buffer, supported by a single high-bandwidth memory system for all ports. However, as the port scale expands to tens or even hundreds, this architecture faces two core limitations: First, the access bandwidth of the centralized buffer must scale to $2N \times \text{line rate}$ as the port count N increases, making it difficult for the on-chip memory system to maintain sufficient throughput under area and power constraints. Second, integrating the $O(N^2)$ switching network and control logic onto a single large die leads to high power density and significantly reduces manufacturing yield[16] in advanced processes. Constrained by these physical and process limitations, the industry is gradually shifting to a more scalable modular design paradigm, such as the tile-based[11, 12, 33] or chiplet architectures[31]. By partitioning and physically distributing switching and buffering resources into modules, such architectures can enhance system scalability, power efficiency, and manufacturing yield, all while sustaining high throughput.

However, a key architectural feature of the modular design paradigm is that the previously centralized shared buffer is physically partitioned and distributed among independent tiles. This buffer fragmentation exposes its limitations under imbalanced traffic (e.g., incast or hotspot). Specifically, the static physical isolation hinders global resource sharing: when traffic concentrates on a few hotspot nodes, their local buffers quickly saturate and trigger backpressure, leading to a reduced capacity to absorb bursts and a loss of bandwidth; meanwhile, buffers at non-hotspot nodes remain idle, reducing global buffer utilization and system throughput[2]. To address this problem, the goal of this research is: to overcome the physical isolation of distributed buffers at an acceptable cost and to achieve dynamic and effective global buffer sharing and management.

To overcome the problem of low buffer utilization caused by physical isolation, we first propose a Region-based Buffer Sharing Framework. This framework introduces a small number of dedicated interconnects that physically link previously isolated distributed buffer units and logically unify them into a manageable sharing domain. Within each domain, any shared buffer can be accessed on demand by multiple neighboring tiles, enabling effective pooling with controlled wiring and area overhead.

Based on this framework, we introduce CrediX, a credit-driven distributed buffer management mechanism that leverages the existing on-chip Credit-Based Flow Control (CBFC)[12] used for lossless delivery. Rather than adding new flow-control channels, CrediX exploits the cycle-level credit feedback already maintained for each upstream VC. A decrease in private credits directly signals active buffer consumption, enabling cycle-accurate demand sensing without additional coordination overhead. Based on this real-time demand, the Regional Credit Allocator (RCA) dynamically grants

*Corresponding author: renfy@tsinghua.edu.cn

This work is supported in part by National Natural Science Foundation of China (NSFC) under Grant No. 62132007, No. 62221003, and No. 62372363 as well as gifts from Huawei and Broadcom.



This work is licensed under a Creative Commons Attribution 4.0 International License. *DAC '26, Long Beach, CA, USA*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2254-7/2026/07

<https://doi.org/10.1145/3770743.3804045>

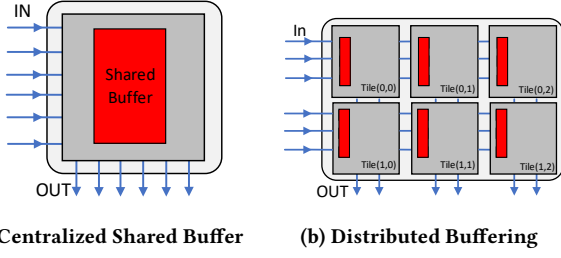


Figure 1: Evolution of switching chip buffering architectures.

buffer usage—conveyed through credits—to the upstream that require them most, thereby enabling on-demand sharing across nodes within a pooling domain.

Unlike request-grant-based reservation schemes[6, 21, 29, 30], this credit-driven allocation avoids round-trip latency and eliminates pipeline stalls, while achieving significantly higher utilization of the shared buffer pool. To address the potential packet reordering introduced by the multi-path nature of pooled buffers, CrediX integrates a dedicated VC-to-Path mapping to guarantee strict in-order delivery. All control logic is implemented locally within each tile, coordinating through the existing NoC fabric, which keeps hardware complexity low and naturally supports modular scaling to larger many-core systems.

We evaluate CrediX using BookSim2[20], a cycle-accurate NoC simulator simulating an 8×8 tile-based switching chip with a Flattened Butterfly topology. CrediX reduces backpressure by up to 65.6% for synthetic Hotspot workloads and improves buffer utilization. Under a realistic workload, it balances load across tiles, lowering peak buffer occupancy from 97%.

2 Background and Motivation

2.1 Buffering Architectures in Switching Chips

In modern high-performance switching chips, the Centralized Shared Buffer remains a widely adopted architectural paradigm[1, 32, 35]. This architecture achieves high statistical multiplexing efficiency and overall buffer utilization under bursty traffic by mapping all port queues to a single, global on-chip SRAM, eliminating the need for per-port statically allocated buffers. However, as port radix and link bandwidth continue to increase, this architecture exposes limitations in scalability. On one hand, the single, centralized SRAM must serve all port read/write requests simultaneously, requiring its access bandwidth to scale to $2N \times \text{line rate}^1$ for an N -port switch, which places extreme demands on the on-chip memory system. On the other hand, fabricating such a global, high-bandwidth SRAM module in advanced process nodes becomes a critical manufacturing bottleneck: this large monolithic block consumes excessive die area and is highly susceptible to fabrication defects, which severely reduces manufacturing yield[3, 16]. These challenges make centralized shared buffers increasingly unsuitable for next-generation AI/HPC switching chips.

To overcome these scalability limitations, Distributed Buffering (DB)[2, 9, 25] has become the promising paradigm, as shown in

Figure 1b. This approach enhances scalability by physically distributing buffering and switching logic across modular, scalable *Tile* (e.g., ‘Tile(0,0)’) or chiplet designs. In this design, arbitration logic is decentralized, and the corresponding buffers are collocated. While this modularity is the mainstream approach for modern NoCs and high-end switch chips, Figure 1b clearly reveals the new problem: because the buffers are physically isolated within each tile, the system suffers from uneven resource utilization under imbalanced traffic. At its core, the limitation of distributed buffering is the difficulty of efficiently sharing resources across tiles.

2.2 Buffering in Distributed Architectures

In a Tile/Chiplet distributed design, buffers and switching logic are physically distributed across various tiles. These tiles are interconnected via an efficient on-chip network (NoC), such as a Flattened Butterfly (FB)[23] or a 2D Mesh topology. Among all buffer types in this architecture, the input buffers are the dominant category in both capacity and performance impact.

Input buffers absorb incoming traffic by storing packets to optimize system throughput. In practice, they are typically implemented with multiple Virtual Channels (VCs²)[10] to avoid Head-of-Line (HoL) blocking. This category accounts for the largest capacity[25, 33], consuming the majority of the total on-chip buffer space. However, when an input buffer fills up, it triggers Priority-based Flow Control (PFC) to ensure lossless delivery. PFC pauses only the affected priority class, but the mechanism is coarse-grained and reactive: once a pause is issued, the upstream tile must halt transmission for an entire pause interval, even if congestion quickly disappears. This conservative behavior suppresses available link bandwidth and leads to underutilization. Moreover, all flows within the paused priority class are stalled, including those destined for uncongested outputs, causing intra-priority Head-of-Line (HOL) Blocking [4]. As a result, frequent PFC events—common when distributed input buffers experience uneven load—can significantly degrade effective throughput[17].

2.3 Credit-Based Flow Control (CBFC)

In off-chip interconnects (e.g., Ethernet), PFC is commonly employed to prevent packet loss. Operating at priority granularity and exhibiting long pause-resume cycles, PFC is well suited for centimeter- to meter-scale links but is less appropriate for latency-sensitive on-chip communication. Within the NoC of modern switching chips[18, 22], Credit-Based Flow Control (CBFC)[24] is the predominant mechanism. As illustrated in Figure 2, the downstream port tracks its available buffer slots and returns a credit to the upstream port through a Credit Generator whenever a flit³ is consumed. The upstream maintains a corresponding Credit Counter (shown with a value of “2” in the figure) and transmits a flit only when the counter is positive. This forms a hop-by-hop closed loop: sending a flit consumes one credit, while buffer release at the downstream replenishes it. With flit-level feedback and per-VC granularity, CBFC provides precise, stable, and low-overhead flow regulation for distributed switching architectures.

²A Virtual Channel (VC) is a logical queue abstraction that shares the physical link bandwidth and buffer with other VCs.

³“Flit” (Flow control unit) is the atomic unit of bandwidth allocation and flow control. A packet is typically segmented into multiple fixed-length flits for transmission.

¹Line rate denotes the nominal maximum data transmission rate that the port’s physical bus (or link) can provide.

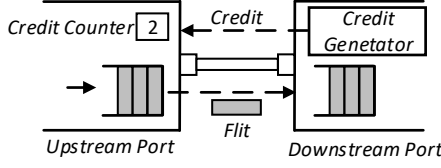


Figure 2: Hop-by-hop Credit-based flow control (CBFC).

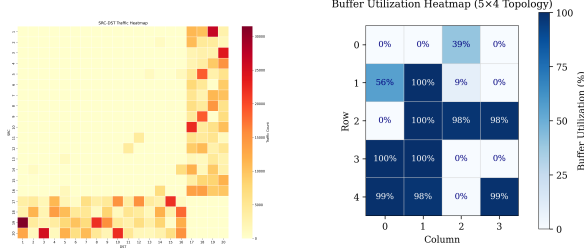


Figure 3: Data center leaf-switch traffic load heatmap. Figure 4: Baseline 5x4 topology buffer-usage heatmap.

2.4 Motivation

Traffic in modern data centers and Networks-on-Chip (NoCs) is often non-uniform. Figure 3 shows a traffic load heatmap from a Alibaba data center leaf switch[27], which clearly illustrates that a few ports receive a concentrated amount of traffic while other ports remain relatively idle. This imbalanced traffic pattern leads to severe performance bottlenecks: buffers at some ports become heavily congested, while buffer resources elsewhere are left unused.

In a distributed buffering (DB) architecture, such spatial imbalance causes some tiles' buffers to saturate while others remain idle, reducing overall buffer efficiency and throughput. To visualize this problem at the chip-architecture level, we conducted a baseline experiment in BookSim2[20]. We configured an 5×4 NoC with a Flattened Butterfly (FB) interconnection, using the standard on-chip CBFC flow control mechanism. The buffer and VC configurations were set to reference the YARC architecture[26] (similar to Cray's 64-port switch[33]). We then applied a real-world traffic load from an Alibaba data center leaf switch in a Clos architecture. Figure 4 shows heatmaps of traffic load and buffer occupancy. A clear spatial imbalance is visible: dark blue areas (e.g., 'Row 1, Col 1' and most of 'Row 2') show buffers at 98%-100% occupancy, while light areas (e.g., most of 'Row 0' and 'Row 3') are almost completely empty (0% occupancy).

This observation clearly demonstrates that even with sufficient total on-chip buffer capacity, the physical isolation of distributed buffers leads to significant resource waste and performance bottlenecks under non-uniform traffic. Based on this observation, our goal is to design a new buffer management mechanism to achieve higher effective buffer utilization, reduce congestion-triggered stalls (like PFC) under imbalanced loads, and ultimately improve end-to-end throughput.

3 Framework

The core idea of our design is to implement a Region-based Buffer Sharing mechanism. We posit that this solution achieves a practical

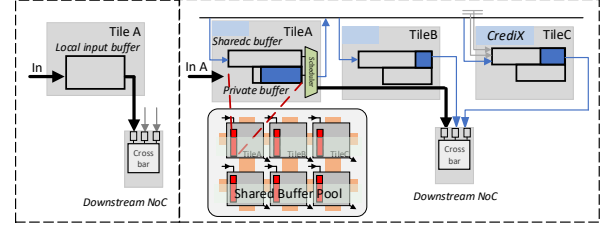


Figure 5: Region-based buffer sharing framework vs. traditional architecture.

trade-off between the high utilization efficiency but costly overhead of Global Sharing architectures and the inefficiency of Isolated Buffering designs.

Figure 5 illustrates this contrast in architecture. The existing architecture (left side) shows a tile with a single "Local input buffer," where all incoming data traverses this solitary path to the "Downstream NoC". In contrast, our framework (right side) introduces three key techniques:

- **Buffer Partitioning:** First, we physically partition the local buffer of each tile (e.g., TileA) into a Private (solid blue) and a Shared (white) portion.
- **Region-based Pooling:** As shown in the bottom diagram of Figure 5, we define a "Region"—for instance, a row of physically adjacent tiles (TileA, TileB, TileC). All "Shared" buffers within this region are then organized into a logically unified Shared Buffer Pool, as visualized by the "equivalent" dashed-line block at the top.
- **Centralized Management:** This unified shared pool is managed by our CrediX mechanism, which acts as a centralized credit manager, as indicated by the arrows pointing from the "CrediX" block to the shared buffers.

Our architecture establishes two parallel forwarding paths for data packets arriving at any input port (e.g., "In A") within a tile. The Private Path (indicated by the thick black arrow in Figure 5) utilizes the dedicated "Private" buffer. This path provides a guaranteed, low-latency baseline channel. Crucially, as it operates independently of the dynamic shared pooling and allocation logic, it ensures stable pipeline progress and incurs no additional coordination overhead or latency penalties under low congestion conditions. Concurrently, the Shared Path (thin blue arrow) utilizes the dynamically available "Shared" buffer pool. Crucially, the Shared Path employs a Direct Forwarding mechanism. Unlike conventional borrowing schemes, which incur latency and channel overhead by requiring packets to return to the source before transmission, our design forwards shared packets directly to the "Downstream NoC." This direct transfer significantly reduces overall packet latency and minimizes channel occupancy on the network fabric.

However, as highlighted by the red dashed lines in Figure 5, this dual-path shared architecture also introduces two critical design challenges that we must solve:

- **Shared Management Policy:** This is not just about deciding which path a packet takes. The deeper, essential challenge is: how should we dynamically allocate the credits for this shared buffer

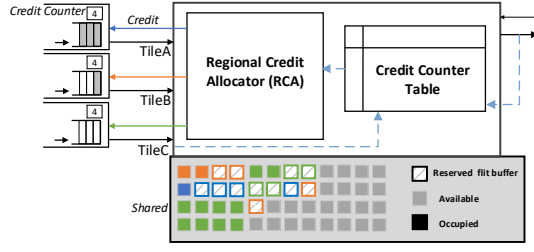


Figure 6: Overview of the CrediX mechanism.

pool among multiple competing flows? We must design an efficient and fair policy (the core CrediX algorithm) to simultaneously achieve both high utilization and fairness.

- **Architectural Integration and Consistency:** This challenge is twofold. First, **Ordering Consistency:** if flits from the same data flow traverse both the private path (black line) and the shared path (blue line), they may arrive at the "Downstream NoC" out of order. A mechanism must be designed to strictly guarantee packet order. Second, **Defining the Shared Domain:** we must decide how (e.g., Row-based) and why to partition the shared scope, to find a balance between performance gains and implementation overhead.

4 CrediX Design

4.1 Demand-Based Dynamic Credit Allocation

CrediX is an enhanced design built upon the standard CBFC framework. As illustrated in Figure 6, it introduces a shared buffer pool dynamically managed by a new component, the Regional Credit Allocator (RCA). This enables real-time awareness of upstream congestion and facilitates on-demand allocation. Strategically, CrediX reuses the existing credit return path and adjusts the credit allocation logic to achieve dynamic management, all without requiring additional signaling overhead.

CrediX's dynamic allocation capability relies on a "private reservation + dynamic sharing" buffer structure. The system reserves a fixed-capacity buffer (the 'Reserved flit buffer') defined by a threshold θ_t for each upstream port to ensure fairness and pipeline stability; the remaining flit buffers form a dynamic shared pool. The RCA (the control box in Figure 6) acts as the centralized dispatcher for this shared pool, managing dynamic allocation via two key types of state registers in Credit Counter Table: a global shared counter ocr (tracking the total number of allocated credits from the shared pool), and a per-port in-flight credit register group R (where each element R_i maintains the number of credits allocated to, but not yet consumed by, upstream port i).

The core idea of CrediX is to perform a on-demand best-effort allocation after observing actual demand. In each cycle, the RCA monitors the private credit status of all VCs from the Credit Counter Table. Whenever it detects a decrease in a VC's private credits, which indicates that the upstream VC is actively consuming its buffer, it triggers the allocation algorithm. Algorithm 1 implements a minimal, best-effort replenishment policy. To ensure fair access and prevent monopolization of the shared pool, the RCA employs

Algorithm 1 Demand-Based Dynamic Credit Allocation (Round-Robin, Best-Effort)

```

1:  $ocr$ : global over-credit counter
2:  $R[i]$ : in-flight shared credits of upstream VC  $i$ 
3:  $i \leftarrow \text{rand}() \bmod N_{vc}$  ▷ random starting VC
4: for  $k = 1$  to  $N_{vc}$  do ▷ one full round-robin sweep
5:    $\text{capacityOK} \leftarrow (N_{vc} \cdot \theta_t + ocr) < msize$ 
6:    $\text{thresholdOK} \leftarrow R[i] < \theta_t$ 
7:   if  $\text{capacityOK}$  and  $\text{thresholdOK}$  then
8:      $\text{SEND\_CREDIT}(i)$ 
9:      $R[i] \leftarrow R[i] + 1$ 
10:     $ocr \leftarrow ocr + 1$  ▷ issue one credit to VC  $i$  if needed
11:   end if
12:    $i \leftarrow (i + 1) \bmod N_{vc}$ 
13: end for
14: return ▷ end of one RR sweep; next cycle will repeat

```

two key fairness controls: Round-Robin Scheduling guarantees every active VC receives periodic service, and a per-VC threshold check ($R[i] < \theta_t$) limits the maximum number of shared credits an individual VC can hold. When there are available credits in the global shared pool, the RCA scans each VC in a round-robin manner and supplies shared credits to active VCs as needed, allocating as many as possible until either the pool is exhausted or the VC no longer requires additional credits. It is worth emphasizing that the CrediX management mechanism is designed to be flexible and configurable. This means that common on-chip shared buffer strategies (e.g., DT[5], etc.) can be seamlessly integrated and deployed on this framework. Algorithm 1 represents the most basic implementation to illustrate CrediX's working principle. Finally, the RCA updates the upstream VC's shared credits and synchronously updates ocr and R_i according to the algorithm's decisions.

To quantitatively analyze the theoretical gain of this on-demand mechanism under 'Incast' traffic, we formalize it. The equivalent available buffer B_{credix} is a function of the port buffer B_{port} , private ratio P_r , total Incast nodes k_{total} , total tiles N^2 , shared domain size n , and private reservation θ_t :

$$B_{credix} = (B_{port} \cdot P_r) + n\theta_t + \frac{N^2}{k_{total}} (B_{port}(1 - P_r) - n\theta_t) \quad (1)$$

This formula (Equation 1) provides the key guiding significance: CrediX's gain primarily comes from the "leverage factor" $\frac{N^2}{k_{total}}$, which is maximized when k_{total} is small (high imbalance). It also reveals the "total reservation cost" of $n\theta_t$, mathematically proving why a trade-off (Region-based) n must be chosen over global sharing.

4.2 Ordering and Regional Partitioning

After addressing the core allocation mechanism of CrediX, we now resolve the second key challenge identified in Section 3: implementation details. This primarily involves two aspects: guaranteeing ordering consistency and defining the shared domain.

Guaranteeing Ordering Consistency. A critical challenge is the reordering risk: in our dual-path model where private and shared buffers coexist, packets from the same flow traveling both paths concurrently could arrive at the destination out of order. To strictly

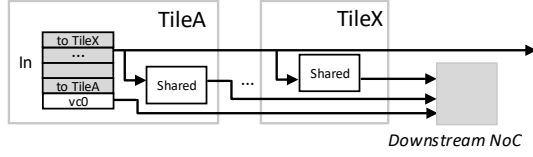


Figure 7: Dedicated VC-to-Path mapping for in-order delivery.

resolve this, our solution is "Dedicated VC-to-Path Mapping." As shown in Figure 7, the core of this scheme is to establish a strict, fixed mapping between the multiple Virtual Channels (VCs) at the input port 'In' of 'TileA' and the physical buffer paths. We designate *vc0* as exclusively dedicated to the "Private Path"; its arrow bypasses all 'Shared' buffer blocks and goes directly to the 'Downstream NoC'. All remaining VCs (VCs 1-R) are exclusively dedicated to the "Shared Path," traversing one or more 'Shared' blocks. The key to consistency is Flow-to-VC Binding: when a data flow (identified by its Flow ID) enters the 'In' port, our mechanism permanently binds it to a single, specific VC for its entire duration. Order is thus guaranteed at the hardware level. Because all packets from a single flow are strictly confined to the same VC, and each VC itself is a strict FIFO queue, this ensures all packets for a flow follow only one path, eliminating the possibility of reordering.

Defining the Shared Domain. The second critical implementation question is how to partition the "Shared Domain." A global domain (i.e., all tiles sharing one pool) would introduce unacceptable wiring complexity and management overhead. Therefore, as mentioned in Section 3, we adopt region-based sharing. Our approach is to leverage the physical topology to minimize implementation overhead. Specifically, one implementation is to group tiles that are physically located in the same row (e.g., 'TileA', 'TileB', and 'TileC') into the same shared domain. This row-based partitioning decision is based on two benefits: first, in a tile-based layout, tiles within the same row are physically closest, minimizing wiring length and latency; second, this partitioning method aligns naturally with the routing patterns of many modern NoC topologies, such as the Flattened Butterfly (FB), thereby simplifying the forwarding paths for both data and credits.

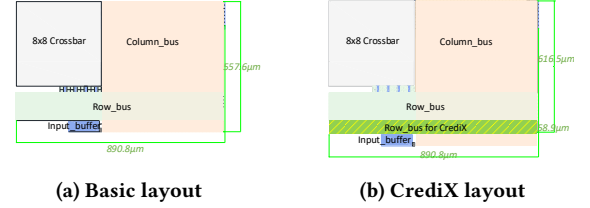
4.3 Overhead Analysis

To quantitatively evaluate the incremental overhead of CrediX, we adopt an analysis method based on existing public hardware parameters. Architecturally, we select the industry-recognized Cray BlackWidow YARC router (90nm)[33] as our baseline. This router is similar to our design, also using an 8x8 Tile array, and its detailed microarchitectural data (e.g., 11.32M Logic cells and 12.98M Register cells) provide a reliable reference for our analysis. On the process technology front, we align our analysis with mainstream 102.4T switch chips (e.g., Tomahawk 6[19]), placing our evaluation in the context of an advanced 3nm process[7], using the public parameters shown in Table 1, to assess its feasibility.

First, the storage overhead is minimal. CrediX does not modify the core buffer sizes, thus incurring no additional SRAM overhead. The only storage cost comes from the registers for the 'Credit Counter Table' and the *ocr* counter. For a Row-based shared domain

Table 1: Key 3nm-based Process Parameters for Overhead Estimation.

Parameter	Value	Description
SRAM bit-cell size (μm^2)	0.021	Public Data [7]
Interconnect pitch (nm)	23	Public Data [7]
Data path bit-width	640	readily achievable with current HBM technology[8]
Data path width (μm)	14.72	$0.023 \times 640 = 14.72$



(a) Basic layout

(b) CrediX layout

Figure 8: Floorplan comparison of a single tile.

with 8 tiles, assuming $\theta_t = 16$ (requiring 4 bits per entry), the total register overhead per tile is only 40 bits ($8 \times 4 + 8$). The total overhead for a 64-tile chip is 2560 bits, which is negligible compared to the YARC baseline's 12.98M register cells. The logic overhead is also negligible, stemming primarily from the 8 sets of 4-bit adders/subtractors, one 8-bit adder/subtractor for counter updates, and the RCA's comparison logic. We estimate this totals approximately 600 logic gates. Compared to the YARC router's 11.32M logic cells, this overhead has almost no practical impact.

Interconnect wiring is the dominant area overhead. In Fig. 8, CrediX introduces lightweight flit pathways (Row_bus for CrediX) between buffers to support on-demand cross-node provisioning. Under the double-frequency assumption, each pathway has an effective width of 320 bits; when evaluated under a full-bandwidth configuration, the upper bound of the associated area increase is approximately 10.6%. Although these pathways are capable of carrying flits, their load is significantly lower than that of the Row_bus: intra-region credit dispatch is only triggered under localized congestion and is naturally rate-limited by the sharing mechanism. Consequently, these pathways do not need to match the full bandwidth of the main link and can be further narrowed, leading to an actual wiring cost lower than the theoretical upper bound. Overall, under a 3 nm process, an area increase on the order of 10% represents a reasonable and engineering-practical tradeoff for the performance benefits enabled by CrediX.

5 Evaluation

5.1 Experimental Setup

We implemented our CrediX mechanism within BookSim2[20], a widely accepted cycle-accurate Network-on-Chip (NoC) simulator. We simulated an 8x8 tile-based switching chip, utilizing a Flattened Butterfly (FB) as the NoC interconnection topology. This topological choice aligns with our Design discussion regarding the shared domain: to align with the minimal wiring overhead in FB topology, we adopted a row-based shared region partitioning scheme. Key

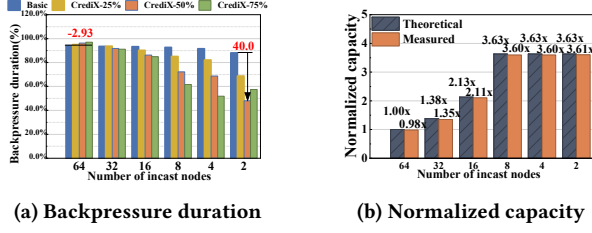


Figure 9: Incast traffic evaluation.

microarchitectural parameters were configured as follows: a clock frequency of 300 MHz, on-chip link bandwidth of 1 flit/cycle, a flit width of 640 bits, XY routing, and Round-robin allocation.

To ensure a fair comparison, we referenced HIER (YARC-like) settings[26], configuring the total on-chip buffer to 2MB (averaging 36.25kB per tile). In all configurations, each port's input buffer contains 2 VCs. We evaluated the following two port buffer schemes, which have identical total capacities: (1) Basic: A traditional distributed architecture where each port has a 256-flit private input buffer. (2) CredIX: Our design, where each port's 256-flit total capacity is partitioned, we partition it by a default (and adjustable) ratio of 50%. This setup guarantees that any performance improvement is a direct result of CredIX's management efficiency, not an increase in total buffer resources.

To comprehensively evaluate CredIX, we used both synthetic and real-world traffic workloads. The synthetic traffic was modeled as fixed-length flows: each flow consists of 10 packets, and each packet contains 4 flits, for a total flow length of 40 flits. Our target traffic patterns were carefully crafted to reproduce the non-uniform pressure scenarios discussed in the Motivation section, including typical Incast (many-to-one) and Hotspot (a many-to-one pattern with random skew). Furthermore, we used real-world traffic traces from data centers[27] to validate the practical effectiveness of our solution.

5.2 Synthetic Workloads

We evaluate CredIX using Incast (many-to-one) and Hotspot traffic, analyzing the impact of the shared buffer ratio (25%-75%). All simulations were run for 4000 cycles.

Incast Evaluation: At $k = 64$ (global congestion), no idle buffers are shareable, and CredIX-25% incurs a slight overhead (Figure 9a). However, as k decreases, traffic becomes imbalanced, CredIX's advantage is significant, and a larger shared ratio provides better mitigation. At $k = 2$ (high imbalance), CredIX-75% reduces backpressure by 40.0 percentage points. This is because CredIX achieves its theoretical capacity gain: Figure 9b shows the measured capacity (3.60x) almost perfectly matches the theoretical (3.63x) defined by Equation (1).

Hotspot Evaluation: In the Hotspot experiment, we select an x ratio of 16 sources as "hotspot senders" (to one destination) and the rest as "background senders" (to random non-hotspot destinations). At $x = 0.125$ (high imbalance), the Basic scheme suffers 86.9% backpressure (Figure 10a). CredIX-75% cuts this to 29.9%, a 65.6% relative reduction. Figure 10(b) reveals the reason: Basic utilization is only 3.2% due to fragmentation, while CredIX-75% uses the shared pool to absorb bursts, reaching 34.5% utilization. Even at $x = 1$

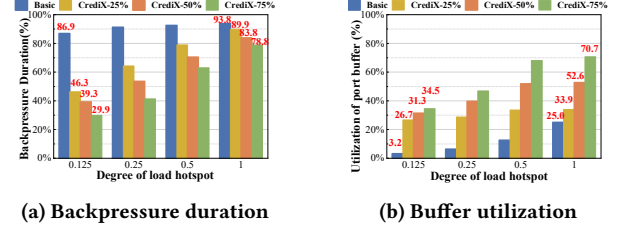


Figure 10: Hotspot traffic evaluation.

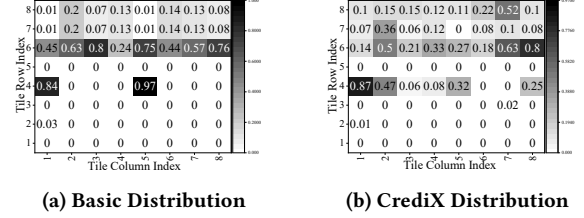


Figure 11: Buffer occupancy heatmaps under the HPCC trace at 2000 cycles.

(all 16 nodes are hotspots), CredIX still outperforms Basic by using resources from the 48 other idle nodes in the 8×8 system (Figure 10).

5.3 Real-World Traffic Traces

To validate CredIX's effectiveness in a realistic scenario, we employed a highly non-uniform data center traffic trace from HPCC[27], injecting it into an 8×8 tile system and capturing buffer occupancy at 2000 cycles. The heatmap for the Basic scheme in Figure 11(a) reproduces the core problem: the system exhibits significant load polarization. Severely congested hotspots (e.g., 0.97 occupancy in Row 4) coexist with large idle regions (e.g., Rows 1, 3, and 5), indicating that Basic struggles to effectively utilize available capacity under real-world traffic due to buffer fragmentation.

After adopting CredIX (50% shared), Figure 11(b) shows that the peak single-node buffer utilization is reduced, and the load distribution is significantly more balanced. The RCA's intra-row management redistributes local congestion pressure to neighboring resources. The results demonstrate that CredIX can effectively balance load, alleviate hotspots, and improve overall resource utilization under real-world traffic.

6 Conclusion

In this paper, we propose CredIX, a dynamic buffer management mechanism for large-scale distributed switching chips that leverages a region-based shared buffer framework. At its core, the Regional Credit Allocator (RCA) allocates shared buffer pool on demand while a dedicated VC-to-Path mapping preserves packet order. By dynamically adapting to non-uniform traffic, CredIX improves buffer utilization and reduces port backpressure, as demonstrated in both synthetic Hotspot workloads and real HPCC traces. These results show that CredIX can achieve high performance with moderate engineering cost, providing a practical solution for efficient, adaptive buffer management in large-scale NoCs.

References

- [1] Anurag Agrawal and Changhoon Kim. 2020. Intel tofino2—a 12.9 tbps p4-programmable ethernet switch. In *2020 IEEE Hot Chips 32 Symposium (HCS)*. IEEE Computer Society, 1–32.
- [2] Matthias A Blumrich, Nan Jiang, and Larry R Dennison. 2018. Exploiting idle resources in a high-radix switch for supplemental storage. In *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 45–57.
- [3] Joo-Hyung Chae. 2024. High-bandwidth and energy-efficient memory interfaces for the data-centric era: Recent advances, design challenges, and future prospects. *IEEE Open Journal of the Solid-State Circuits Society* (2024).
- [4] Wenxue Cheng, Kun Qian, Wanchun Jiang, Tong Zhang, and Fengyuan Ren. 2020. Re-architecting congestion management in lossless ethernet. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. 19–36.
- [5] Abhijit K Choudhury and Ellen L Hahne. 2002. Dynamic queue length thresholds for shared-memory packet switches. *IEEE/ACM Transactions On Networking* 6, 2 (2002), 130–140.
- [6] Nikolaos Chrysos. 2006. *Request-grant scheduling for congestion elimination in multistage networks*. Ph. D. Dissertation. Πανεπιστήμιο Κρήτης. Σχολή Θετικών και Τεχνολογικών Επιστημών. Τμήμα
- [7] Wikipedia Contributors. 2025. 3 nm process. https://en.wikipedia.org/wiki/3_nm_process#cite_note-3nm_jedm-57
- [8] Wikipedia Contributors. 2025. 3 nm process. https://en.wikipedia.org/wiki/High_Bandwidth_Memory
- [9] Yi Dai, Kefei Wang, Gang Qu, Liquan Xiao, Dezun Dong, and Xingyun Qi. 2017. A scalable and resilient microarchitecture based on multiport binding for high-radix router design. In *2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 429–438.
- [10] William J Dally et al. 1992. Virtual-channel flow control. *IEEE Transactions on Parallel and Distributed systems* 3, 2 (1992), 194–205.
- [11] William J Dally and Brian Towles. 2001. Route packets, not wires: on-chip interconnection networks. In *Proceedings of the 38th annual design automation conference*. 684–689.
- [12] William James Dally and Brian Patrick Towles. 2004. *Principles and practices of interconnection networks*. Elsevier.
- [13] Sujal Das and Rochan Sankar. 2012. Broadcom smart-buffer technology in data center switches for cost-effective performance scaling of cloud applications. *Broadcom White Paper* (2012).
- [14] Daniele De Sensi, Salvatore Di Girolamo, Kim H McMahon, Duncan Roweth, and Torsten Hoefer. 2020. An in-depth analysis of the slingshot interconnect. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–14.
- [15] Greg Faanes, Abdulla Bataineh, Duncan Roweth, Tom Court, Edwin Froese, Bob Alverson, Tim Johnson, Joe Kopnick, Mike Higgins, and James Reinhard. 2012. Cray cascade: a scalable HPC system based on a Dragonfly network. In *SC'12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–9.
- [16] Yinxiao Feng and Kaisheng Ma. 2022. Chiplet actuary: A quantitative cost model and multi-chiplet architecture exploration. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 121–126.
- [17] Jinbin Hu, Chaoliang Zeng, Zilong Wang, Hong Xu, Jiawei Huang, and Kai Chen. 2022. Load balancing in PFC-enabled datacenter networks. In *Proceedings of the 6th Asia-Pacific Workshop on Networking*. 21–28.
- [18] Broadcom Inc. 2025. Reimagining the Ethernet Switch for HPC and AI Scale-up. Online. <https://www.broadcom.com/company/news/product-releases/63341>
- [19] Broadcom Inc. 2025. StrataXGS BCM78910 Series. <https://www.broadcom.com/products/ethernet-connectivity/switching/strataxgs/bcm78910-series>
- [20] Nan Jiang, Daniel U Becker, George Michelogiannakis, James Balfour, Brian Towles, David E Shaw, John Kim, and William J Dally. 2013. A detailed and flexible cycle-accurate network-on-chip simulator. In *2013 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 86–96.
- [21] Nan Jiang, Daniel U Becker, George Michelogiannakis, and William J Dally. 2012. Network congestion avoidance through speculative reservation. In *IEEE International Symposium on High-Performance Comp Architecture*. IEEE, 1–12.
- [22] Wanchun Jiang, Fengyuan Ren, and Jianxin Wang. 2018. Survey on link layer congestion management of lossless switching fabric. *Computer Standards & Interfaces* 57 (2018), 31–35.
- [23] John Kim, James Balfour, and William Dally. 2007. Flattened butterfly topology for on-chip networks. In *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007)*. IEEE, 172–182.
- [24] NT Kung and Robert Morris. 1995. Credit-based flow control for ATM networks. *IEEE network* 9, 2 (1995), 40–48.
- [25] Cunlu Li, Dezun Dong, Xiangke Liao, and John Kim. 2021. Hybrid memory buffer microarchitecture for high-radix routers. *IEEE Trans. Comput.* 71, 11 (2021), 2888–2902.
- [26] Cunlu Li, Dezun Dong, Xiangke Liao, John Kim, and Changhyun Kim. 2019. DeepHiR: Improving high-radix router throughput with deep hybrid memory buffer microarchitecture. In *Proceedings of the ACM International Conference on Supercomputing*. 403–413.
- [27] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh, et al. 2019. HPCC: High precision congestion control. In *Proceedings of the ACM special interest group on data communication*. 44–58.
- [28] Xiang-Ke Liao, Zheng-Bin Pang, Ke-Fei Wang, Yu-Tong Lu, Min Xie, Jun Xia, De-Zun Dong, and Guang Suo. 2015. High performance interconnect network for Tianhe system. *Journal of Computer Science and Technology* 30, 2 (2015), 259–272.
- [29] Zhonghai Lu and Axel Jantsch. 2008. TDM virtual-circuit configuration for network-on-chip. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 16, 8 (2008), 1021–1034.
- [30] Behnam Montazeri, Yilong Li, Mohammad Alizadeh, and John Ousterhout. 2018. Homa: A receiver-driven low-latency transport protocol using network priorities. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*. 221–235.
- [31] Samuel Naffziger, Noah Beck, Thomas Burd, Kevin Lepak, Gabriel H Loh, Mahesh Subramony, and Sean White. 2021. Pioneering chiplet technology and design for the amd epyc™ and ryzen™ processor families: Industrial product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 57–70.
- [32] NVIDIA. 2022. NVIDIA Announces Spectrum High-Performance Data Center Networking Infrastructure Platform. <https://nvidia-driv.nvidia.com/news/nvidia-announces-spectrum-high-performance-data-center-networking-infrastructure-platform>
- [33] Steve Scott, Dennis Abts, John Kim, and William J Dally. 2006. The blackwidow high-radix cros network. *ACM SIGARCH Computer Architecture News* 34, 2 (2006), 16–28.
- [34] Shaohuai Shi, Zhenheng Tang, Xiaowen Chu, Chengjian Liu, Wei Wang, and Bo Li. 2020. A quantitative survey of communication optimizations in distributed deep learning. *IEEE Network* 35, 3 (2020), 230–237.
- [35] Bob Wheeler. 2019. Tomahawk 4 Switch First to 25.6Tbps. Broadcom Doubles 400Gbps Ports With Unprecedented 512 Serdes. *Microprocessor Report* (Dec 2019). <https://docs.broadcom.com/doc/12398014>
- [36] Min Xie, Yutong Lu, Kefei Wang, Lu Liu, Hongjia Cao, et al. 2011. Tianhe-1a interconnect and message-passing services. *IEEE Micro* 32, 1 (2011), 8–20.